Weight Adaptation In RL-Assisted MPC

2026-04-01

Method Goal

This method uses a TD3 agent to adapt the four scalar penalties that define the MPC stage cost: two output-tracking weights and two input-move weights. The continuous policy does not replace the MPC optimizer. It changes the optimizer's relative priorities online by multiplying the baseline Q_{out} and R_{in} entries.

Notebook Sources Covered

- `RL_assisted_MPC_weights.ipynb`
- `RL_assisted_MPC_weights_dist.ipynb`
- Shared helpers from `TD3Agent/agent.py` and `Simulation/mpc.py`

Shared MPC Context

The baseline MPC stays fixed in structure:

- the plant still receives the first move of an optimization-based MPC,
- the observer state still drives the internal prediction model,
- the RL policy only changes how expensive different errors and moves are.

If the baseline penalties are

$$Q_{\text{out}}^{(0)} = [Q_1, Q_2], \qquad R_{\text{in}}^{(0)} = [R_1, R_2],$$

then the agent chooses a four-vector of multipliers

$$m_t = [m_{Q_1}, m_{Q_2}, m_{R_1}, m_{R_2}]$$

and the active MPC penalties become

$$Q_{\text{out}}^{(t)} = [Q_1 m_{Q_1}, Q_2 m_{Q_2}], \qquad R_{\text{in}}^{(t)} = [R_1 m_{R_1}, R_2 m_{R_2}].$$

State, Action, Reward, And Update Logic

The state is the same scaled observer-state bundle used by most TD3 notebook families:

$$s_t = \begin{bmatrix} \tilde{x}_t \\ \tilde{y}_{\mathrm{sp},t} \\ \tilde{u}_t^{\mathrm{dev}} \end{bmatrix}.$$

The action is continuous with dimension four. A raw TD3 action $a_t \in [-1, 1]^4$ is remapped into user-defined bounds:

$$m_t = \ell + \frac{a_t + 1}{2} \odot (h - \ell),$$

where ℓ and h are the lower and upper coefficient bounds.

The reward still comes from closed-loop tracking and move quality after solving the weighted MPC problem. This notebook family also multiplies the reward by 0.01 before storing it.

Core Algorithm Flow

1. Build the scaled RL state from the observer state and operating point.
2. Query the TD3 agent for a four-dimensional action.
3. Map that action to four penalty multipliers.
4. Update Q_{out} and R_{in} inside the current MPC object.
5. Solve MPC, apply the first move to the plant, and roll the observer.
6. Store the transition in the TD3 replay buffer and call `train_step()` after warm start.

Key Code Paths

Notebook-local penalty remapping

```
Q_base = np.array([Q1_penalty, Q2_penalty], dtype=float)
R_base = np.array([R1_penalty, R2_penalty], dtype=float)

def map_to_bounds(a, low, high):
    return low + ((a + 1.0) / 2.0) * (high - low)

def set_penalties(MPC_obj, Q_base, R_base, m):
    Q1_new = float(Q_base[0] * m[0])
    Q2_new = float(Q_base[1] * m[1])
    R1_new = float(R_base[0] * m[2])
    R2_new = float(R_base[1] * m[3])
    MPC_obj.Q_out = np.array([Q1_new, Q2_new])
    MPC_obj.R_in = np.array([R1_new, R2_new])
```

Notebook-local TD3 control loop

```
if i > WARM_START:
    if not test:
        action = agent.take_action(current_rl_state, explore=(not test))
    else:
        action = agent.act_eval(current_rl_state)
else:
    action_ = np.array([1., 1., 1., 1.])
    action = map_from_bounds(action_, low_coef, high_coef)

m = map_to_bounds(action, low_coef, high_coef)
set_penalties(MPC_obj, Q_base, R_base, m)

sol = spo.minimize(
    lambda x: MPC_obj.mpc_opt_fun(x, y_sp[i, :],
                                  scaled_current_input_dev,
                                  xhatdhat[:, i]),
    IC_opt, bounds=bnds, constraints=cons
)
```

Shared TD3 interaction and update logic

```
@torch.no_grad()
def take_action(self, state, explore=False):
    self.steps += 1
    s = torch.as_tensor(state, dtype=torch.float32, device=self.device)
    a = self.actor(s).detach().cpu().numpy()
    if explore:
        self._expl_sigma = self.expl_sched.value(self.steps)
        a = a + np.random.randn(*a.shape) * self._expl_sigma
    return np.clip(a, -self.max_action, self.max_action)

def train_step(self):
    ...
    next_action = (base_next + noise).clip(-self.max_action, self.max_action)
    q_next = self.critic_target.combined_forward(ns, next_action,
                                                 mode=self.target_combine)
    y = r + self.gamma * (1.0 - done) * q_next
```

Variants Covered

- The nominal notebook adapts weights without injected process drift.
- The disturbance notebook applies gradual changes to process quantities while reusing the same TD3-based weight tuning loop.
- Both notebooks log the active weight multipliers over time through `mult_log`.

Limitations, Caveats, And Open Questions

- This family assumes the MPC structure is correct and only tunes the cost.
- The weight bounds are notebook-level configuration, not a shared schema.
- The control loop is still notebook-local, so refactors must preserve the notebook supervisor behavior instead of editing only module helpers.
- Historical comparisons depend on preserving the same baseline Q and R definitions.